

Dyck Symmetric Functions

1 Status

This note records the status of Dyck symmetric functions in the curated repository. The classical case is proved in the 2026 preprint *Dyck Symmetric Functions and Applications to q, t -Catalan Polynomials*. The proof first establishes the dual case by an explicit tableau insertion algorithm and then derives the affine, or nondual, case from the dual case using the standard involution on symmetric functions.

The $r = st + 1$ analogue below is conjectural. The accompanying code checks systematic finite parameter boxes for both the affine and dual identities.

2 The $r = st + 1$ Setup

Fix a nonnegative integer step parameter t . For any finite integer sequence $x = (x_0, \dots, x_{\ell-1})$, set

$$\text{dinv}_t(x) = \sum_{0 \leq i < j < \ell} d_t(x_i, x_j),$$

where

$$d_t(a, b) = \begin{cases} \max(0, a + t - b), & a \leq b, \\ \max(0, b + 1 + t - a), & a > b. \end{cases}$$

An *affine rational Dyck sequence of step t* is a finite integer sequence satisfying

$$x_{i+1} \leq x_i + t \quad \text{for all } i.$$

A *dual rational Dyck sequence of step t* is a finite integer sequence satisfying

$$x_{i+1} > x_i + t \quad \text{for all } i.$$

Let S be a finite multiset of integers. A factorization of S is a sequence of finite words whose concatenation is a rearrangement of S . It is affine, respectively dual, if each factor is an affine, respectively dual, rational Dyck sequence of step t . Define

$$\text{DS}^{(t)}(S, d; \mathbf{x}) = \sum_{\substack{\mathcal{F} \text{ affine factorization of } S \\ \text{dinv}_t(F_0 F_1 F_2 \dots) = d}} x^{\mathcal{F}}$$

and

$$\mathrm{DS}^{*(t)}(S, d; \mathbf{x}) = \sum_{\substack{\mathcal{F} \text{ dual factorization of } S \\ \mathrm{dinv}_t(F_0 F_1 F_2 \cdots) = d}} x^{\mathcal{F}}.$$

A rational Dyck tableau of step t is a left-aligned tableau whose rows are dual rational Dyck sequences of step t , and whose columns, read from bottom to top, are affine rational Dyck sequences of step t . Let $\lambda(P)$ be its shape and let $\mathrm{RR}(P)$ be its row-reading word.

3 Classical Theorem

Theorem 1 (Classical dual Dyck symmetric functions). *For every finite multiset S and every $d \geq 0$, the classical dual Dyck symmetric function satisfies*

$$\mathrm{DS}^*(S, d; \mathbf{x}) = \sum_P s_{\lambda(P)}(\mathbf{x}),$$

where P ranges over classical Dyck tableaux with entries S and $\mathrm{dinv}(\mathrm{RR}(P)) = d$.

Remark 1. *The proof in the 2026 preprint is constructive. It gives an explicit insertion algorithm sending dual Dyck factorizations to Dyck tableaux together with semistandard recording data. The local row insertion is iterated through the rows of a tableau, and the reverse insertion proves bijectivity.*

Theorem 2 (Classical affine Dyck symmetric functions). *For every finite multiset S and every $d \geq 0$, the classical affine Dyck symmetric function satisfies*

$$\mathrm{DS}(S, d; \mathbf{x}) = \sum_P s_{\lambda(P)'}(\mathbf{x}),$$

where the indexing set P is the same set of classical Dyck tableaux.

Remark 2. *In the preprint, this nondual statement is derived from the dual statement by comparing the fundamental-quasisymmetric expansions and applying the standard involution ω , which sends s_λ to $s_{\lambda'}$.*

4 Conjectural $r = st + 1$ Analogue

Conjecture 1 (Rational Dyck symmetric functions, $r = st + 1$). *Let S be a finite multiset of integers, let $d \geq 0$, and fix $t \geq 0$. Then*

$$\mathrm{DS}^{(t)}(S, d; \mathbf{x}) = \sum_P s_{\lambda(P)'}(\mathbf{x}),$$

where P ranges over rational Dyck tableaux of step t whose entries are exactly S and whose row-reading word satisfies

$$\mathrm{dinv}_t(\mathrm{RR}(P)) = d.$$

Similarly,

$$\mathrm{DS}^{*(t)}(S, d; \mathbf{x}) = \sum_P s_{\lambda(P)}(\mathbf{x}),$$

with the same indexing set of rational Dyck tableaux.

Remark 3. For $t = 1$, this specializes to the classical theorem. For $t > 1$, this is currently treated as a conjecture. The directory `code/` includes systematic finite checks over bounded multisets and all occurring `dinv` values for $t = 2, 3, 4$, and includes the classical insertion algorithm code used for the theorem-level case.